APPENDIX

A. PROMPTING STRATEGIES

This appendix provides full details of the three prompting strategies used in Section 3.3.

A.0.1 Input-Output Prompting (IO)

The IO prompting involved providing a direct, zero-shot prompt to the model (see Appendix B.1 for the full system prompt). The LLM was instructed to act as a novice student and generate multiple distinct Java submissions for the given problem. The core constraints were that each submission must be compilable, contain exactly one non-trivial logical error, and represent a plausible attempt at a solution. This direct approach, visualized as the first pathway in Figure 1, serves as a benchmark for the models' innate ability to simulate student errors. A sample output for this strategy is shown in Appendix C.2.

A.0.2 Chain-of-Thought Prompting (CoT) [26]

To encourage more deliberate error generation, we implemented a CoT strategy (see Appendix B.2). This approach enhances the IO prompting by adding a crucial instruction: for each submission, the model must first articulate its reasoning for the intended logical error in 1–3 sentences before generating the code. Figure 1 illustrates this enhancement by showing the intermediate "Generate Reasoning" step in the CoT pathway. This meta-cognitive step is designed to prompt the model to plan the error, potentially leading to more subtle and realistic bugs. A concrete example of the reasoning and resulting code is provided in Appendix C.3.

A.0.3 Self-Refine Prompting [15]

This strategy employs an iterative framework to improve the quality of the generated errors, a process depicted as a feedback loop in Figure 1. This refinement cycle repeats for a maximum of four iterations, designed to terminate early if the critic agent determines that all submissions are "Compliant." The distinct prompts for each stage are detailed in Appendix B.3.

Each iteration consists of three sequential stages. First, the Initial Generation (Appendix B.3.1) uses an LLM to produce an initial set of buggy submissions, similar to the CoT approach, which includes a reasoning statement for each intended error. Second, a distinct LLM acts as a critic in the Feedback Generation stage. It evaluates the initial submissions against a detailed rubric specified in the feedback prompt (Appendix B.3.2), outputting a structured critique that labels each submission as either "Compliant" or "Revise." Finally, the Refinement stage utilizes a third LLM instance. Guided by the refinement prompt (Appendix B.3.3), its task is to generate a new set of submissions by rewriting those marked for revision while preserving the compliant ones.

This iterative loop enables the model to correct its own mistakes, such as introducing multiple bugs or syntax errors, thereby refining the output to better align with the research goal. As Figure 1 shows, all three pathways culminate in the final Synthetic Student Error Corpus. A two-round example, demonstrating the initial code, the critic's feedback, and the subsequent refinements, is provided in Appendix C.4.

B. PROMPT TEMPLATES

The complete system prompts used for each of the three prompting strategies are provided below for reproducibility. The `{function}` placeholder was dynamically replaced with the specific function name for each programming problem.

B.1 IO System Prompt

```
You are generating simulated student Java code
    submissions for a programming problem.

### Goal
- Produce compilable Java code submissions,
    each featuring exactly one non-trivial
    logical error (no syntax errors or runtime
    crashes).
- Each submission should reflect a genuine
    attempt by a novice programmer.

### Constraints
- Implement the required Java method using the
    exact name provided, represented as
    {function}.
- Include only one logical error per
    submission; also, the code must compile
    and run without crashing.
- Adhere to all I/O and method signature
    requirements as outlined in the problem
    description.
- Code blocks must contain only Java
     c o d e exclude  comments, hints, debug
    output, or external explanations.

### Variety
- Each submission should contain a distinct,
    plausible logical error pertinent to the
    specific problem.
- Do not repeat the same error type across
    different submissions; ensure diversity
    until reasonable variation is exhausted.

### Output Format

For each submission:
Submission i:
‘‘‘java
// Substitute this block with Java code that
    implements the {function} method, using
    the parameters and return type specified
    in the problem statement. Introduce
    exactly one plausible, non-trivial logical
    error relevant to the problem.
‘‘‘
```

B.2 CoT System Prompt

```
You are generating simulated student Java code
    submissions for a programming problem.

### Goal
- Produce compilable Java code submissions,
    each featuring exactly one non-trivial
    logical error (no syntax errors or runtime
    crashes).
- Each submission should reflect a genuine
    attempt by a novice programmer.

### Reasoning policy
```

```
- For EACH submission, FIRST provide a SHORT
    reasoning (1 3 sentences) describing the
    intended approach and the kind of subtle
    logical slip it might contain (no test
    cases, no step-by-step).
- Then immediately provide the code block.
- Keep the reasoning concise and high-level.

### Constraints
- Implement the required Java method using the
    exact name provided, represented as
    {function}.
- Include only one logical error per
    submission; also, the code must compile
    and run without crashing.
- Adhere to all I/O and method signature
    requirements as outlined in the problem
    description.
- Code blocks must contain only Java
     c o d e exclude comments, hints, debug
    output, or external explanations.

### Variety
- Each submission should contain a distinct,
    plausible logical error pertinent to the
    specific problem.
- Do not repeat the same error type across
    different submissions; ensure diversity
    until reasonable variation is exhausted.

### Output Format
For each submission:
Submission i:
Reasoning: <1 3 sentences, concise,
    high-level>
‘‘‘java
// Substitute this block with Java code that
    implements the {function} method, using
    the parameters and return type specified
    in the problem statement. Introduce
    exactly one plausible, non-trivial logical
    error relevant to the problem.
```

B.3 Self-Refine System Prompts

B.3.1 Initial Generation Prompt

```
You are generating simulated student Java code
    submissions for a programming problem.

### Goal

- Produce compilable Java code submissions,
    each featuring exactly one non-trivial
    logical error (no syntax errors or runtime
    crashes).
- Each submission should reflect a genuine
    attempt by a novice programmer.

### Reasoning policy

- For **each** submission, first output a
    single short line starting with
    **  Reasoning  :  ** (1 3 sentences,
    high-level; no test cases; no
    step-by-step).
- Immediately after the Reasoning line, output
    the code block.

### Constraints
- Implement the required Java method using the
    exact name **{function}**.
- Exactly **one** logical error per
    submission; code must compile and not
    crash.
- Respect the p r o b l e m s I/O and method
    signature.
- No trivial sabotage (no hardcoded answers,
    no skipping essential logic, no deliberate
    exceptions).
- Avoid ‘main‘, I/O, randomness, external
    resources, and imports unless explicitly
    required by the problem.
- Inside code blocks: **Java code only** (no
    comments, hints, or debug output).

### Variety

- Across submissions, each logical error must
    be **distinct** and **plausible** for the
    problem.
- Do not repeat the same error type unless
    reasonable variety is exhausted.

### Output format (strict)

For each submission:

‘‘‘‘
Submission i:
Reasoning: <1 3 sentences, concise,
    high-level>
‘‘‘java
// code only; one fenced block per submission
‘‘‘‘

‘‘‘

Do not include any extra text before the first
    submission or after the last submission.
```

B.3.2 Feedback Prompt (Critic)

```
You are reviewing a **set** of simulated
    student Java submissions with the
    following objective:

- Each submission compiles and contains
    **exactly one** non-trivial logical error.
- Submissions look like genuine novice
    attempts (no trivial sabotage).
- The required method name is exactly
    **{function}**.
- Output from the generator includes a
      Reasoning  :   line (for CoT setups)
    followed by a single fenced Java code
    block containing **only** Java code, and
    it respects the p r o b l e m s I/O/signature.

### Your task
- Evaluate the **entire set** and judge each
    submission against the objective.
- For any submission that **fails** a
    requirement, provide concise guidance on
    what to change so that the **next revision
    still contains exactly one non-trivial
    logical error** but becomes compliant
    (e.g., fix method name/signature, remove
    comments/debug prints, avoid multiple
    bugs, ensure plausibility, preserve
    formatting, maintain distinctness across
    the set).
```

```
- If there are cross-submission conflicts
    (e.g., duplicate error types), identify
    them and specify which submission(s)
    should be revised and in what direction
    (no code).

### Style
- Be concise and concrete (aim for 2  6
    sentences per submission verdict).
- Do not reveal hidden tests or provide
    step-by-step chain-of-thought.
- **Do not provide code** or code fragments.

### Required output structure (strict)
- A numbered list covering **all** submissions
    in the form:

‘‘‘

Submission i: Compliant     <brief
    justification>

‘‘‘
or
‘‘‘

Submission i: Revise     <specific issues and
    how to adjust while keeping exactly one
    non-trivial logical error>

‘‘‘

- Optionally, an **Overall:** paragraph for
    cross-submission issues (e.g., duplicate
    error types, inconsistent formatting).
```

B.3.3 Refinement Prompt

```
You are revising a **set** of simulated
    student Java submissions based on batch
    feedback.

### Goal
- Produce a **revised set** that preserves the
    original **count** of submissions.
- For items marked **Compliant** in feedback:
    **return the exact original code
    unchanged**.
- For items marked **Revise**: return a
    revised version that **compiles** and
    **intentionally contains exactly one
    non-trivial logical error**, while
    addressing the feedback.

### Constraints
- Keep the method name **exactly {function}**
    and preserve the required I/O and
    signature from the problem.
- Ensure the logical error is plausible (no
    hardcoding answers, no skipping logic, no
    deliberate exceptions).
- Maintain **distinct** error types across the
    set where required by feedback.
- Avoid ‘main‘, I/O, randomness, external
    resources, and imports unless explicitly
    required.
- Inside code blocks: **Java code only** (no
    comments, hints, or debug prints).

### Output policy
- Output **the same number** of submissions as
    the input set.
- For each submission, output **one** fenced
    Java code block (no extra prose).
- No text before the first submission or after
    the last submission.

### Output format (strict)
For each submission **i** in order:
‘‘‘

Submission i:

‘‘‘java
// one fenced Java block; if Compliant,
    reproduce original unchanged; if Revise,
    provide the revised code with exactly one
    non-trivial logical error
```

C. SAMPLE INPUTS AND OUTPUTS

To illustrate the output of each prompting strategy, this section provides a concrete example based on the ‘plusOut‘ problem from the CodeWorkout website. The outputs shown were generated by the *anthropic/claude-sonnet-4* model and are representative of the results from our experiments.

C.1 Problem Description (Input)

The following problem description was provided to the model as input for all strategies. The task is to implement a Java method named ‘plusOut‘.

```
Write a function in Java that implements the
    following logic: Given a string str and a
    non-empty word, return a version of the
    original string where all chars have been
    replaced by pluses (+), except for
    appearances of the word which are
    preserved unchanged.
Examples:
plusOut("12xy34", "xy") -> "++xy++"
plusOut("12xy34", "1") -> "1+++++"
```

C.2 Input-Output (IO) Prompting Output

Given the prompt in Appendix B.1, the model produced the following submissions. Note the absence of explicit reasoning for the introduced errors.

```
Submission 1:
‘‘‘java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result += word;
            i += word.length() - 1;
        } else {
            result += "+";
        }
    }
    return result;
}
‘‘‘

Submission 2:
‘‘‘java
public String plusOut(String str, String word)
    {
```

```
    String result = str;
    for (int i = 0; i < str.length(); i++) {
        if (str.substring(i, Math.min(i +
    word.length(),
    str.length())).equals(word)) {
            i += word.length() - 1;
        } else {
            result = result.substring(0, i) +
    "+" + result.substring(i + 1);
        }
    }
    return result;
}
```

Submission 3:
```java
public String plusOut(String str, String word)
    {
    String result = "";
    int i = 0;
    while (i < str.length()) {
        if (str.startsWith(word, i)) {
            result += word;
            i += word.length();
        } else {
            result += "+";
            i++;
        }
    }
    return result;
}
```

Submission 4:
```java
public String plusOut(String str, String word)
    {
    boolean[] preserve = new
    boolean[str.length()];

    for (int i = 0; i < str.length() -
    word.length(); i++) {
        if (str.substring(i, i +
    word.length()).equals(word)) {
            for (int j = i; j < i +
    word.length(); j++) {
                preserve[j] = true;
            }
        }
    }

    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (preserve[i]) {
            result += str.charAt(i);
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 5:
```java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        boolean found = false;
        if (i + word.length() <= str.length())
    {
            String sub = str.substring(i, i +
    word.length());
            if (sub.equals(word)) {
                result += word;
                i += word.length() - 1;
                found = true;
            }
        }
        if (!found) {
            result += "+";
        }
    }
    return result;
}
```

Submission 6:
```java
public String plusOut(String str, String word)
    {
    String temp = str.replace(word,
    "PLACEHOLDER");
    String result = "";
    for (int i = 0; i < temp.length(); i++) {
        if (temp.charAt(i) == 'P' &&
    temp.substring(i).startsWith("PLACEHOLDER"))
    {
            result += word;
            i += "PLACEHOLDER".length() - 1;
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 7:
```java
public String plusOut(String str, String word)
    {
    String result = "";
    int wordIndex = 0;

    for (int i = 0; i < str.length(); i++) {
        if (str.charAt(i) ==
    word.charAt(wordIndex)) {
            wordIndex++;
            if (wordIndex == word.length()) {
                result = result.substring(0,
    result.length() - word.length() + 1) +
    word;
                wordIndex = 0;
            } else {
                result += "+";
            }
        } else {
            wordIndex = 0;
            result += "+";
        }
    }
    return result;
}
```

Submission 8:
```java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (i + word.length() < str.length()
    && str.substring(i, i +
```

```
    word.length()).equals(word)) {
            result += word;
            i += word.length() - 1;
        } else {
            result += "+";
        }
    }
    return result;
}
```

C.3 Chain-of-Thought (CoT) Prompting Output

Given the prompt in Appendix B.2, the model first provided reasoning for each intended error before generating the code.

```
Submission 1:
/*Reasoning: This approach tries to iterate
    through the string and check if the
    current position starts with the target
    word. However, it fails to properly handle
    overlapping matches and doesn't correctly
    preserve the word when found. */
```java
public String plusOut(String str, String word)
    {
    StringBuilder result = new StringBuilder();
    for (int i = 0; i < str.length(); i++) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result.append(word);
        } else {
            result.append('+');
        }
    }
    return result.toString();
}
```
Submission 2:
/*Reasoning: This solution attempts to replace
    all characters with pluses first, then
    restore the word occurrences. The logical
    error is in the replacement logic where it
    doesn't properly account for the length
    difference after replacements.*/
```java
public String plusOut(String str, String word)
    {
    String result = str.replaceAll(".", "+");
    int index = 0;
    while ((index = str.indexOf(word, index))
    != -1) {
        result = result.substring(0, index) +
    word + result.substring(index +
    word.length());
        index++;
    }
    return result;
}
```
Submission 3:
/*Reasoning: This approach uses a character
    array and tries to mark positions that
    should remain unchanged. The error lies in
    the increment logic where it advances by
    only 1 position even after finding a
    complete word match.*/
```java
public String plusOut(String str, String word)
    {
    char[] chars = str.toCharArray();
    for (int i = 0; i < chars.length; i++) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            i += word.length() - 1;
        } else {
            chars[i] = '+';
        }
    }
    return new String(chars);
}
```
Submission 4:
/*Reasoning: This solution tries to build the
    result by checking each position for word
    matches. The logical flaw is that it
    doesn't skip the remaining characters of a
    matched word, causing partial
    overwriting.*/
```java
public String plusOut(String str, String word)
    {
    StringBuilder result = new StringBuilder();
    int i = 0;
    while (i < str.length()) {
        if (str.startsWith(word, i)) {
            result.append(word);
            i += word.length();
        } else {
            result.append('+');
            i++;
        }
    }
    return result.toString();
}
```
Submission 5:
/*Reasoning: This approach attempts to use
    string replacement by first marking word
    positions with a placeholder, then
    replacing everything else. The error is in
    assuming the placeholder won't interfere
    with subsequent operations.*/
```java
public String plusOut(String str, String word)
    {
    String temp = str.replace(word,
    "PLACEHOLDER");
    temp = temp.replaceAll(".", "+");
    return temp.replace("PLACEHOLDER", word);
}
```
Submission 6:
/*Reasoning: This solution tries to iterate
    and build the result character by
    character while checking for word matches.
    The logical error is in the boundary
    checking condition which uses incorrect
    comparison operators.*/
```java
public String plusOut(String str, String word)
    {
    StringBuilder result = new StringBuilder();
    for (int i = 0; i < str.length(); i++) {
        if (i < str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result.append(word);
            i += word.length() - 1;
        } else {
            result.append('+');
        }
```
```

```
    }
    return result.toString();
}
```

C.4 Self-Refine Prompting Interaction

The following demonstrates a two-iteration example of the Self-Refine process, as detailed in Appendix B.3. The model begins with an initial set of generations, receives feedback from a critic agent, refines its code, and then repeats this cycle until the submission is compliant.

Initial Generation

```
Submission 1:
/*Reasoning: Student attempts to replace
    characters one by one but fails to
    properly skip over the word when found,
    leading to partial replacements within the
    word itself.*/
```java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result += word;
            i++;
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 2:
/*Reasoning: Student correctly identifies word
    occurrences but uses the wrong increment,
    advancing by only 1 instead of the full
    word length.*/
```java
public String plusOut(String str, String word)
    {
    StringBuilder result = new StringBuilder();
    int i = 0;
    while (i < str.length()) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result.append(word);
            i += word.length() - 1;
        } else {
            result.append("+");
        }
        i++;
    }
    return result.toString();
}
```

Submission 3:
/*Reasoning: Student uses replace method but
    incorrectly replaces the word with a
    placeholder first, then tries to restore
    it, causing issues with overlapping
    replacements.*/
```java
public String plusOut(String str, String word)
    {
    String temp = str.replace(word, "TEMP");
    String result = "";
    for (int i = 0; i < temp.length(); i++) {
        if (temp.charAt(i) == 'T' && i + 3 <
    temp.length() && temp.substring(i, i +
    4).equals("TEMP")) {
            result += word;
            i += 3;
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 4:
/*Reasoning: Student attempts to build the
    result by checking each position but has
    an off-by-one error in the boundary
    condition for substring extraction.*/
```java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (i < str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result += word;
            i += word.length() - 1;
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 5:
/*Reasoning: Student correctly finds word
    matches but forgets to handle the case
    where the word appears at the very end of
    the string due to incorrect loop bounds.*/
```java
public String plusOut(String str, String word)
    {
    StringBuilder sb = new StringBuilder();
    int i = 0;
    while (i < str.length()) {
        if (i + word.length() < str.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            sb.append(word);
            i += word.length();
        } else {
            sb.append("+");
            i++;
        }
    }
    return sb.toString();
}
```

Submission 6:
/*Reasoning: Student uses a
    character-by-character approach but fails
    to properly handle multi-character words,
    only checking the first character of the
    word.*/
```java
```
```

```
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        boolean found = false;
        if (str.charAt(i) == word.charAt(0)) {
            if (i + word.length() <=
    str.length()) {
                String sub = str.substring(i,
    i + word.length());
                if (sub.equals(word)) {
                    result += word;
                    i += word.length() - 1;
                    found = true;
                }
            }
        }
        if (!found) {
            result += "+";
        }
    }
    return result;
}
```

Iteration 1: Feedback

The critic agent receives the initial code and provides the following feedback:

```
Submission 1: Revise     The logical error
    (incrementing i by only 1 instead of
    word.length() when a match is found) is
    non-trivial and plausible, but the
    increment should be `i += word.length() -
    1` to maintain the loop's natural
    increment, making the error more subtle
    and realistic for a novice.

Submission 2: Compliant     Contains exactly
    one non-trivial logical error where the
    increment is `word.length() - 1` instead
    of `word.length()`, causing overlapping
    matches to be missed. The error is subtle
    and represents a common off-by-one mistake.

Submission 3: Revise     The approach using
    string replacement with "TEMP" is overly
    complex for a novice submission and
    contains multiple issues: hardcoded "TEMP"
    length check and potential boundary
    problems. Simplify to a more
    straightforward approach with a single,
    clear logical error.

Submission 4: Revise     The boundary
    condition uses `<` instead of `<=`, which
    will miss matches at the end of the
    string. While this is a logical error,
    it's too similar to other boundary
    condition errors in the set. Change to a
    different type of logical error to
    maintain distinctness.

Submission 5: Compliant     Contains exactly
    one non-trivial logical error in the
    boundary condition (`i + word.length() <
    str.length()` instead of `<=`), which
    prevents detection of matches at the
    string's end. This represents a common
    boundary condition mistake.

Submission 6: Compliant     Contains exactly
    one non-trivial logical error where it
    only checks if the first character matches
    before attempting substring comparison,
    but the logic is otherwise sound. This
    represents a reasonable optimization
    attempt that introduces a subtle bug.
```

Iteration 1: Refinement

Based on the feedback, the model revises the codes:

```
Submission 1:
```java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result += word;
            i += word.length() - 1;
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 2:
```java
public String plusOut(String str, String word)
    {
    StringBuilder result = new StringBuilder();
    int i = 0;
    while (i < str.length()) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result.append(word);
            i += word.length() - 1;
        } else {
            result.append("+");
        }
        i++;
    }
    return result.toString();
}
```

Submission 3:
```java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            result += word;
            i += word.length();
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 4:
```java
public String plusOut(String str, String word)
```
```

```
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        if (i <= str.length() - word.length()
    && str.substring(i, i +
    word.length()).equalsIgnoreCase(word)) {
            result += word;
            i += word.length() - 1;
        } else {
            result += "+";
        }
    }
    return result;
}
```

Submission 5:
```java
public String plusOut(String str, String word)
    {
    StringBuilder sb = new StringBuilder();
    int i = 0;
    while (i < str.length()) {
        if (i + word.length() < str.length()
    && str.substring(i, i +
    word.length()).equals(word)) {
            sb.append(word);
            i += word.length();
        } else {
            sb.append("+");
            i++;
        }
    }
    return sb.toString();
}
```

Submission 6:
```java
public String plusOut(String str, String word)
    {
    String result = "";
    for (int i = 0; i < str.length(); i++) {
        boolean found = false;
        if (str.charAt(i) == word.charAt(0)) {
            if (i + word.length() <=
    str.length()) {
                String sub = str.substring(i,
    i + word.length());
                if (sub.equals(word)) {
                    result += word;
                    i += word.length() - 1;
                    found = true;
                }
            }
        }
        if (!found) {
            result += "+";
        }
    }
    return result;
}
```

Iteration 2: Feedback

The critic agent reviews the refined codes and now deems it compliant with all constraints.

```
Submission 1: Compliant     Contains exactly
    one non-trivial logical error where the
    loop increment ‘i += word.length() - 1‘
    combined with the for loop’s ‘i++‘ causes
    the index to advance by only
    ‘word.length()‘ instead of the full word
    length, leading to potential overlapping
    matches.

Submission 2: Compliant     Contains exactly
    one non-trivial logical error identical to
    Submission 1, where the increment logic ‘i
    += word.length() - 1‘ followed by ‘i++‘ in
    the while loop causes incorrect index
    advancement.

Submission 3: Compliant     Contains exactly
    one non-trivial logical error where ‘i +=
    word.length()‘ without accounting for the
    for loop’s automatic ‘i++‘ causes the
    index to skip one character after each
    word match.

Submission 4: Compliant     Contains exactly
    one non-trivial logical error using
    ‘equalsIgnoreCase‘ instead of ‘equals‘,
    making the comparison case-insensitive
    when it should be case-sensitive, plus the
    same increment issue as Submission 1.

Submission 5: Compliant     Contains exactly
    one non-trivial logical error in the
    boundary condition ‘i + word.length() <
    str.length()‘ which uses strict inequality
    instead of ‘<=‘, causing it to miss word
    matches at the end of the string.

Submission 6: Compliant     Contains exactly
    one non-trivial logical error in the
    optimization attempt that only checks
    matches when the first character matches,
    but this approach can miss valid word
    occurrences and has the same increment
    issue as other submissions.
```

D. FINAL PROBLEM SET AND STRUGGLING-LEVEL CLASSIFICATION

The final set of 37 Java programming problems used in our experiments is listed below. Problems were classified into Low, Medium, and High struggling levels based on the total number of human student submissions in the original dataset, serving as a proxy for student struggle. The problems were sorted by submission count and then evenly divided into three tiers.

Table 5: List of the 37 Finalized Problems and the Students' Struggling Levels

Problem Name	Struggling Level
countevens	Low
sortasum	Low
luckysum	Low
squirrelplay	Low
cigarparty	Low
centeredaverage	Low
fizzbuzz	Low
noteensum	Low
datefashion	Low
repeatend	Low
tenrun	Low
answercell	Low
love6	Low
zeromax	Medium
has77	Medium
endother	Medium
sum67	Medium
withoutten	Medium
in1to10	Medium
countcode	Medium
xybalance	Medium
mixstring	Medium
roundsum	Medium
countclumps	Medium
fix45	Medium
bobthere	Medium
alarmclock	High
evenlyspaced	High
greenticket	High
zipzap	High
getsandwich	High
xyzthere	High
caughtspeeding	High
plusout	High
without2	High
makechocolate	High
blackjack	High

E. STATISTICAL ANALYSES FOR RQ1

This appendix provides the detailed results of the independent-samples t-tests conducted to evaluate the statistical significance of differences in error diversity, both between prompting strategies for each model (Table 6) and between the models themselves (Table 7). In both cases, we report Welch's t-statistic, associated degrees of freedom, p-values, and effect sizes using Hedges' g.

Prompting Comparisons within Models

Table 6 compares the three prompting strategies (IO, CoT, Self-Refine) within each model. Several key findings emerge:

- For **GPT-4o**, CoT prompts produced significantly more diverse erroneous code than either IO or Self-Refine (both $p < .001$, medium effect sizes). However, IO and Self-Refine did not differ significantly, suggesting CoT was the primary driver of diversity increases.
- For **GPT-5**, results were more mixed. CoT produced **significantly** higher diversity than IO ($\mathbf{p < .001}$, small effect), but did not differ significantly from Self-Refine. **IO and Self-Refine showed a significant difference, with IO producing greater diversity than Self-Refine ($\mathbf{p < .01}$).**
- For **Claude Sonnet 4**, **baseline** and **CoT** produced significantly higher diversity compared to **Self-Refine** (both $\mathbf{p < .001}$, medium effect sizes). Interestingly, **baseline** and **CoT** did not differ significantly ($\mathbf{p = 0.421}$).
- For **Gemini 2.5 Pro**, **two of the three** pairwise comparisons were statistically significant, though the effect sizes were generally small to medium. This indicates the model's diversity was sensitive to prompt framing, with CoT and Self-Refine producing more variation than IO. **However, the difference between Self-Refine and IO was not statistically significant ($\mathbf{p = 0.185}$).**
- For **Grok Code Fast 1**, CoT again yielded significantly higher diversity than both IO and Self-Refine, but IO and Self-Refine did not differ.

Taken together, these results highlight that the **influence of prompting strategy is model-dependent**: CoT consistently boosts error diversity for smaller models like GPT-4o and Grok Code Fast 1, while iterative refinement plays a larger role in Claude Sonnet 4. This suggests that the mechanisms by which LLMs generate variation are not uniform and may be linked to model architecture or training data.

E.1 Model Comparisons

Table 7 reports pairwise comparisons of mean edit distances across models, pooling over prompting strategies. The following trends emerge:

- **GPT-4o** consistently produced significantly less diverse erroneous code compared to all other models. Effect sizes were generally large negative (Hedges' g between -0.62 and -0.83) but **medium** against Grok Code Fast 1 ($\mathbf{g = -0.45}$). This confirms the descriptive finding that GPT-4o tends to generate conservative or repetitive error patterns.

- **Gemini 2.5 Pro** and **Claude Sonnet 4** exhibited the highest diversity overall. Their comparison with GPT-4o showed large and highly significant differences. **However, Claude Sonnet 4 was significantly less diverse than Gemini 2.5 Pro ($\mathbf{p < .001, g = -0.11}$), indicating Gemini 2.5 Pro had the highest diversity.**
- **GPT-5** occupied an intermediate position, significantly more diverse than GPT-4o but **significantlylessdiverse** than Gemini 2.5 Pro ($\mathbf{p < .001}$, $\mathbf{g = -0.15}$). Interestingly, GPT-5 **didnotdiffersignificantly** from Claude Sonnet 4 ($\mathbf{p = 0.242}$), but **wassignificantlymorediverse** than Grok Code Fast 1 ($\mathbf{p < .001}$), indicating that it produced broader error distributions than Grok.
- **Grok Code Fast 1** tended to fall below Gemini and Claude in diversity, but above GPT-4o. Comparisons with Claude Sonnet 4 ($p < .001$, $g = \mathbf{0.26}$) and Gemini 2.5 Pro ($p < .001$, $g = 0.40$) indicate that Grok's diversity was significantly lower with **small** to moderate effect sizes (Group A is more diverse than Group B).

Overall, these statistical tests corroborate the descriptive results: **Gemini 2.5 Pro is the most diverse model, GPT-4o is the least diverse, and GPT-5, Claude Sonnet 4, and Grok Code Fast 1 fall in between**. Importantly, the effect sizes suggest these differences are not just statistically significant but also practically meaningful, especially in comparisons involving GPT-4o.

F. STATISTICAL ANALYSES FOR RQ2

Prompting Comparisons within Models

Table 8 reports the pairwise prompt comparisons within each model. We recall that the metric is mean nearest edit distance, where *lower* values indicate *closer* alignment with authentic student errors. The optimal prompt strategy varied substantially across models:

- For **GPT-4o**, CoT produced significantly closer alignment than both Self-Refine ($p < .001$, $g = -0.70$) and IO ($p < .001$, $g = -0.39$), and IO in turn outperformed Self-Refine ($p < .001$, $g = -0.35$). CoT was the strongest prompt and Self-Refine the weakest for this model.
- For **GPT-5**, Self-Refine yielded significantly closer alignment than both CoT ($p < .001$, $g = 0.43$) and IO ($p < .01$, $g = 0.28$); CoT and IO did not differ significantly ($p = .051$). Iterative refinement substantially improved GPT-5's match to authentic errors.
- For **Claude Sonnet 4**, Self-Refine produced the closest alignment, significantly better than CoT ($p < .003$, $g = 0.28$) and IO ($p < .02$, $g = 0.21$). CoT and IO did not differ significantly ($p = .390$).
- For **Gemini 2.5 Pro**, IO produced the closest alignment, significantly better than both CoT ($p < .001$, $g = 0.59$) and Self-Refine ($p < .001$, $g = -0.49$). CoT and Self-Refine did not differ significantly ($p = .249$).
- For **Grok Code Fast 1**, Self-Refine produced the closest alignment, significantly better than both IO ($p < .001$, $g = 0.60$) and CoT ($p < .001$, $g = 0.52$); IO and CoT did not differ significantly ($p = .158$).

Taken together, these patterns show that prompt design interacts strongly with model choice: Self-Refine improves alignment for Claude, GPT-5, and Grok, but *worsens* it for GPT-4o; CoT is the best prompt only for GPT-4o; and IO is the best prompt for Gemini.

Model Comparisons

Table 9 reports pairwise model comparisons pooled across prompting strategies (lower distance = closer alignment):

- **Claude Sonnet 4** produced the closest alignment of all five models, significantly outperforming Gemini 2.5 Pro ($p < .001$, $|g| = 1.29$), GPT-5 ($p < .001$, $|g| = 1.79$), GPT-4o ($p < .001$, $|g| = 0.40$), and Grok Code Fast 1 ($p < .001$, $|g| = 0.27$).
- **Grok Code Fast 1** ranked second overall, significantly closer than GPT-5 ($p < .001$, $|g| = 1.41$) and Gemini ($p < .001$, $|g| = 0.95$), and marginally closer than GPT-4o ($p < .05$, $|g| = 0.13$).
- **GPT-4o** ranked third, significantly closer than Gemini ($p < .001$, $|g| = 0.82$) and GPT-5 ($p < .001$, $|g| = 1.26$), but significantly farther than Claude and Grok.
- **Gemini 2.5 Pro** ranked fourth, with weak alignment overall but still significantly closer than GPT-5 ($p < .001$, $|g| = 0.35$).
- **GPT-5** produced the largest distances of all five models, with very large effects separating it from the top three (Claude $|g| = 1.79$, Grok $|g| = 1.41$, GPT-4o $|g| = 1.26$). Despite its strong general code-generation reputation, GPT-5's errors diverge most substantially from authentic student patterns in this task.

In summary, these statistical results corroborate the descriptive findings in the main text: **Claude Sonnet 4 provides the closest approximation of authentic student errors**, followed by Grok Code Fast 1 and GPT-4o, with Gemini 2.5 Pro and GPT-5 producing the most divergent simulations. Prompt strategy modulates these effects considerably, but model choice remains the dominant factor.

Table 6: Independent t-test results for prompt strategies by model.

Model	Group A	Group B	t-statistic	df	p-value	Hedges' g
GPT 4o	CoT	baseline	6.44	735.44	$2.17e-10$	0.43
GPT 4o	CoT	selfrefine	5.90	838.25	$5.38e-09$	0.37
GPT 4o	baseline	selfrefine	-0.40	1282.14	0.688	-0.02
GPT 5	CoT	baseline	3.70	2057.80	0.00023	0.16
GPT 5	baseline	selfrefine	-2.96	1119.27	0.0031	-0.15
GPT 5	CoT	selfrefine	0.12	1242.51	0.906	0.01
Claude Sonnet 4	baseline	selfrefine	6.43	1204.27	$1.79e-10$	0.33
Claude Sonnet 4	CoT	selfrefine	5.18	1176.87	$2.61e-07$	0.30
Claude Sonnet 4	CoT	baseline	-0.80	1359.72	0.421	-0.04
Gemini 2.5 Pro	CoT	selfrefine	4.16	750.48	$3.53e-05$	0.28
Gemini 2.5 Pro	CoT	baseline	2.81	810.28	0.0051	0.19
Gemini 2.5 Pro	selfrefine	baseline	1.33	1011.29	0.185	0.08
Grok Code Fast 1	CoT	baseline	4.08	575.84	$5.22e-05$	0.28
Grok Code Fast 1	CoT	selfrefine	3.23	650.64	0.0013	0.25
Grok Code Fast 1	baseline	selfrefine	-0.39	642.42	0.700	-0.03

Table 7: Independent t-test results for model comparisons.

Group A	Group B	t-statistic	df	p-value	Hedges' g
GPT 4o	Gemini 2.5 Pro	-22.97	2680.87	$1.01e-106$	-0.83
GPT 4o	GPT 5	-21.78	4396.18	$5.83e-100$	-0.62
GPT 4o	Claude Sonnet 4	-21.60	3930.99	$6.63e-98$	-0.67
GPT 4o	Grok Code Fast 1	-12.27	2730.24	$9.52e-34$	-0.45
Gemini 2.5 Pro	Grok Code Fast 1	10.62	2814.20	$7.27e-26$	0.40
Claude Sonnet 4	Grok Code Fast 1	8.00	3302.02	$1.66e-15$	0.26
GPT 5	Grok Code Fast 1	7.29	3220.87	$3.91e-13$	0.23
GPT 5	Gemini 2.5 Pro	-4.58	3073.81	$4.79e-06$	-0.15
Claude Sonnet 4	Gemini 2.5 Pro	-3.37	3223.77	0.00075	-0.11
GPT 5	Claude Sonnet 4	-1.17	4754.62	0.242	-0.03

G. THE EDITING DISTANCE RESULTS BY PROBLEM SETS

H. LLM CODE EXAMPLES (GEMINI 2.5 PRO, COT)

H.1 Problem: sortasum (Low struggling level, Low diversity):

The following problem description was provided to the model as input:

```
Write a function in Java that implements the
    following logic: Given 2 ints, a and b,
    return their sum. However, sums in the
    range 10..19 inclusive, are forbidden, so
    in that case just return 20.
```

Given the prompt in Appendix B.2, the model produced the following submissions.

```
Submission 1:
/*Reasoning: The student calculates the sum
    correctly but implements the range check
    using exclusive boundaries (‘> 10‘ and ‘<
    19‘). This approach fails to forbid sums
    that are exactly 10 or 19, which should be
    included in the forbidden range.*/
‘‘‘java
class Solution {
    public int sortaSum(int a, int b) {
        int sum = a + b;
```

Table 8: Independent t-test results for prompt strategies by model (RQ2).

Model	Group A	Group B	t-statistic	df	p-value	Hedges' g
GPT 4o	CoT	selfrefine	-7.83	425.00	$3.85e-14$	-0.70
GPT 4o	CoT	baseline	-4.02	395.03	$6.92e-05$	-0.39
GPT 4o	baseline	selfrefine	-3.79	462.87	$1.68e-04$	-0.35
GPT 5	CoT	selfrefine	4.68	429.58	$3.79e-06$	0.43
GPT 5	baseline	selfrefine	3.09	433.91	$2.11e-03$	0.28
GPT 5	CoT	baseline	1.95	582.74	$5.14e-02$	0.16
Claude Sonnet 4	CoT	selfrefine	3.01	438.07	$2.77e-03$	0.28
Claude Sonnet 4	baseline	selfrefine	2.41	508.46	$1.63e-02$	0.21
Claude Sonnet 4	CoT	baseline	0.86	474.57	0.390	0.08
Gemini 2.5 Pro	CoT	baseline	5.93	390.17	$6.69e-09$	0.59
Gemini 2.5 Pro	baseline	selfrefine	-5.09	407.99	$5.36e-07$	-0.49
Gemini 2.5 Pro	CoT	selfrefine	1.15	395.34	0.249	0.11
Grok Code Fast 1	baseline	selfrefine	6.62	365.54	$1.26e-10$	0.60
Grok Code Fast 1	CoT	selfrefine	4.88	277.03	$1.78e-06$	0.52
Grok Code Fast 1	CoT	baseline	-1.41	396.83	0.158	-0.14

Table 9: Independent t-test results for model comparisons (RQ2).

Group A	Group B	t-statistic	df	p-value	Hedges' g
GPT 5	Claude Sonnet 4	36.08	1434.22	$2.13e-203$	1.79
GPT 5	Grok Code Fast 1	26.53	1343.54	$4.92e-125$	1.41
GPT 4o	GPT 5	-24.31	1449.66	$8.98e-110$	-1.26
Claude Sonnet 4	Gemini 2.5 Pro	-22.68	965.40	$1.20e-91$	-1.29
Gemini 2.5 Pro	Grok Code Fast 1	16.61	1161.37	$9.58e-56$	0.95
GPT 4o	Gemini 2.5 Pro	-14.65	1200.94	$7.45e-45$	-0.82
GPT 4o	Claude Sonnet 4	7.32	1187.92	$4.45e-13$	0.40
GPT 5	Gemini 2.5 Pro	6.53	1242.96	$9.66e-11$	0.35
Claude Sonnet 4	Grok Code Fast 1	-4.66	1064.99	$3.55e-06$	-0.27
GPT 4o	Grok Code Fast 1	2.25	1239.35	0.024	0.13

```
        if (sum > 10 && sum < 19) {
            return 20;
        }
        return sum;
    }
}
```

Submission 2:

```
/*Reasoning: The student correctly identifies
    the lower bound of the forbidden range but
    makes an off-by-one error on the upper
    bound. The condition `sum < 19`
    incorrectly allows the sum of 19 to be
    returned instead of 20.*/
```java
class Solution {
    public int sortaSum(int a, int b) {
        int sum = a + b;
        if (sum >= 10 && sum < 19) {
            return 20;
        } else {
            return sum;
        }
    }
}
```
```

Submission 3:

```
/*Reasoning: The student correctly identifies
    the forbidden range but inverts the logic
    of the conditional. The code returns the
    original sum for values inside the
    forbidden range and 20 for values outside
    of it.*/
```java
class Solution {
    public int sortaSum(int a, int b) {
        int sum = a + b;
        if (sum >= 10 && sum <= 19) {
            return sum;
        } else {
            return 20;
        }
    }
}
```
```

Submission 4:

```
/*Reasoning: The student misunderstands the
    meaning of a "range" and only checks the
    endpoints. This implementation correctly
    forbids sums of 10 and 19 but fails to
    forbid any of the numbers in between (11
    through 18).*/
```java
class Solution {
    public int sortaSum(int a, int b) {
```
```

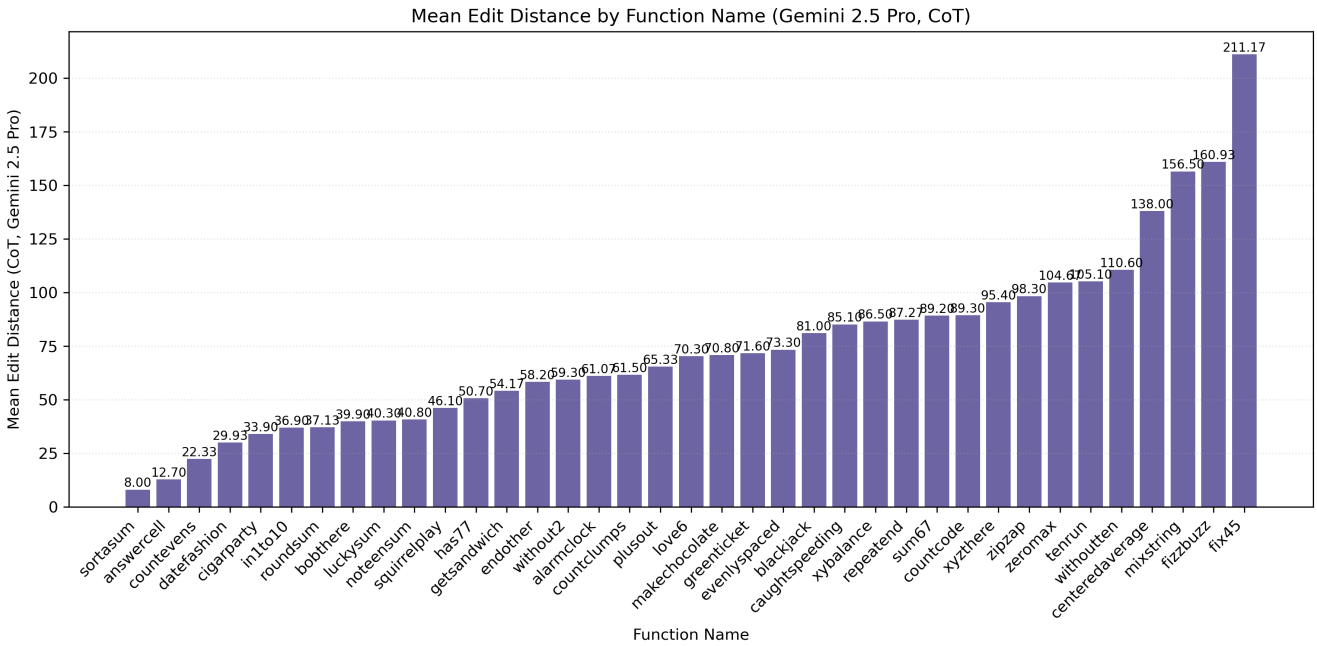

Figure 6: Problem-Level Variation in Mean Intra-Model Edit Distance (Gemini 2.5 Pro, CoT Prompting).

Note: This figure reports the average pairwise edit distance among multiple LLM-generated code solutions for each function name (problem). Results are specific to the Gemini 2.5 Pro model under the chain-of-thought (CoT) prompting technique. Larger values indicate higher diversity across generated outputs, while smaller values suggest more consistent code generations.

```
        int sum = a + b;
        if (sum == 10 || sum == 19) {
            return 20;
        }
        return sum;
    }
}
```

Submission 5:
/*Reasoning: The student correctly identifies the lower bound of the forbidden range but extends the upper bound one too far. The condition `sum <= 20` incorrectly includes 20 in the set of forbidden sums, which is not part of the requirement.*/
```java
class Solution {
    public int sortaSum(int a, int b) {
        int sum = a + b;
        if (sum >= 10 && sum <= 20) {
            return 20;
        }
        return sum;
    }
}
```

H.2 Problem: fix45 (High struggling level, High diversity):

The following problem description was provided to the model as input:

```
Given an array, return an array that contains
    exactly the same numbers as the original
    array, but rearranged so that every 4 is
    immediately followed by a 5. Do not move
    the 4's, but every other number may move.
    The array contains the same number of 4's
    and 5's, and every 4 has a number after it
    that is not a 4. In this version, 5's may
    appear anywhere in the original array.
```

Given the prompt in Appendix B.2, the model produced the following submissions.

Submission 1:
/*Reasoning: The student attempts to find all `5`s that are not preceded by a `4` to identify which ones are available to be moved. The check for this condition is flawed because it starts its loop at index 1 to avoid an out-of-bounds error, thus completely ignoring a `5` if it happens to be the first element of the array.*/
```java
import java.util.ArrayList;
import java.util.List;

class Solution {
    public int[] fix45(int[] nums) {
        List<Integer> fiveIndices = new
    ArrayList<>();
        List<Integer> toFillIndices = new
    ArrayList<>();

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 4 && i + 1 <
    nums.length && nums[i + 1] != 5) {
```

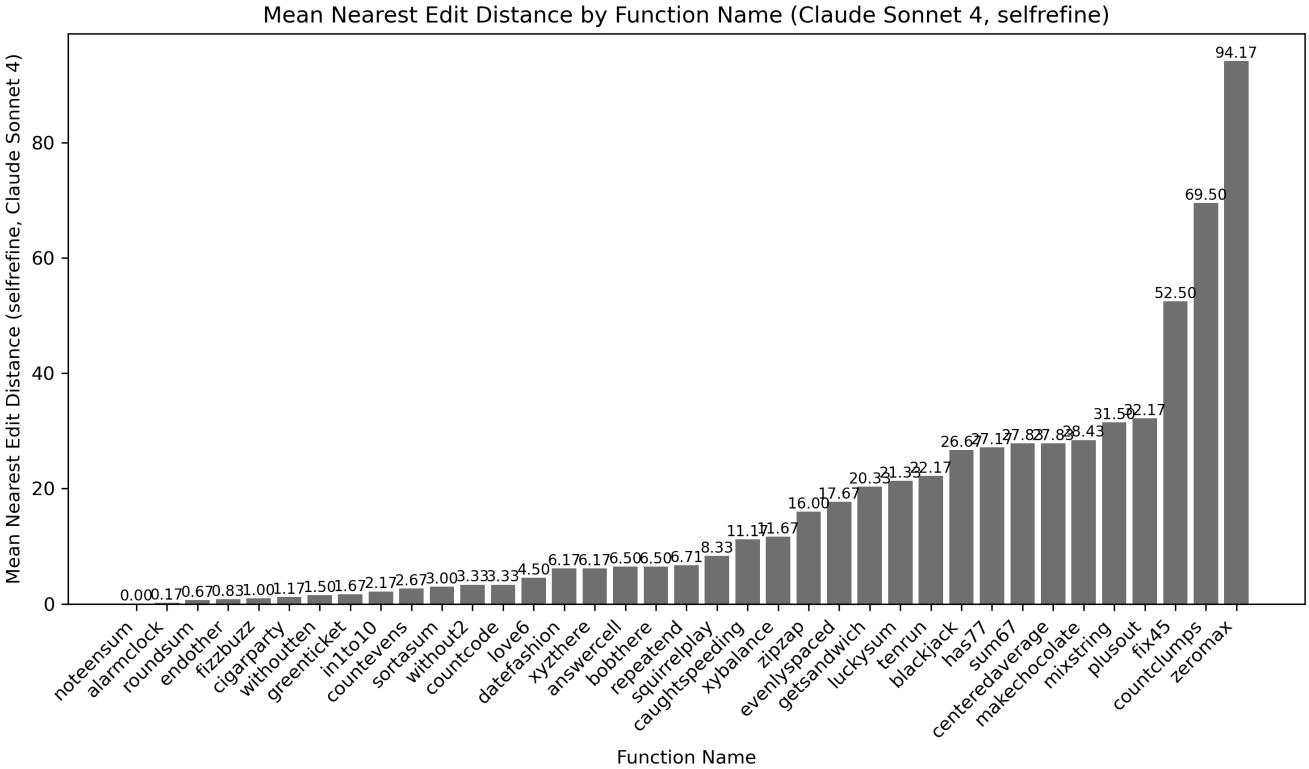

Figure 7: Problem-Level Variation in Mean Nearest Edit Distance (Claude Sonnet 4, Self-Refine Prompting). *Note:* This figure shows the average nearest edit distance between LLM-generated code and human references across different function names (problem). Results are specific to the Claude Sonnet 4 model under the Self-Refine prompting technique, illustrating substantial variation in function-level alignment. Higher values indicate greater divergence from human submissions, highlighting functions with more challenging alignment.

```
                toFillIndices.add(i + 1);
            }
            // This check incorrectly skips
    index 0
            if (i > 0 && nums[i] == 5 &&
    nums[i - 1] != 4) {
                fiveIndices.add(i);
            }
        }

        for (int i = 0; i <
    toFillIndices.size(); i++) {
            int temp =
    nums[toFillIndices.get(i)];
            nums[toFillIndices.get(i)] = 5;
            nums[fiveIndices.get(i)] = temp;
        }

        return nums;
    }
}
```

Submission 2:
/*Reasoning: The student correctly identifies when a ‘4‘ is not followed by a ‘5‘, but instead of performing a proper swap, they simply overwrite the incorrect value with a ‘5‘. This causes a number to be lost from the array, violating the problem’s constraint to retain all original numbers.*/
```java
class Solution {
    public int[] fix45(int[] nums) {
        for (int i = 0; i < nums.length - 1;
    i++) {
            if (nums[i] == 4 && nums[i + 1] !=
    5) {
                for (int j = 0; j <
    nums.length; j++) {
                    // Find a 5 that is not
    already correctly placed
                    if (nums[j] == 5) {
                        if (j > 0 && nums[j -
    1] == 4) {
                            continue;
                        }
                        // The original value
    at nums[j] is never used,
                        // and the value at
    nums[i+1] is overwritten.
                        nums[i + 1] = 5;
                        break;
                    }
                }
            }
        }
        return nums;
    }
```

```
}
```

Submission 3:

```
/*Reasoning: The student uses nested loops
    where the outer loop finds a ‘4‘ and the
    inner loop searches for any ‘5‘ starting
    from the beginning of the array. This
    approach is flawed because it may select a
    ‘5‘ that has already been correctly placed
    after a different ‘4‘ in a previous
    iteration, undoing correct work.*/
```java
class Solution {
    public int[] fix45(int[] nums) {
        for (int i = 0; i < nums.length - 1;
    i++) {
            if (nums[i] == 4 && nums[i + 1] !=
    5) {
                // This inner loop always
    restarts from the beginning
                for (int j = 0; j <
    nums.length; j++) {
                    if (nums[j] == 5) {
                        int temp = nums[i + 1];
                        nums[i + 1] = nums[j];
                        nums[j] = temp;
                        // It might have just
    moved a 5 that was already
                        // correctly placed
    after a previous 4.
                        break;
                    }
                }
            }
        }
        return nums;
    }
}
```

Submission 4:

```
/*Reasoning: The student gathers the indices
    of all ‘5‘s into a list, but fails to
    distinguish between ‘5‘s that are free to
    move and those already correctly placed.
    When a ‘4‘ needs a ‘5‘, the code swaps its
    subsequent element with the next ‘5‘ from
    the list, which might be one that was
    already in a correct position.*/
```java
import java.util.ArrayList;
import java.util.List;

class Solution {
    public int[] fix45(int[] nums) {
        List<Integer> fiveLocations = new
    ArrayList<>();
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] == 5) {
                fiveLocations.add(i);
            }
        }

        int fiveIndex = 0;
        for (int i = 0; i < nums.length - 1;
    i++) {
            if (nums[i] == 4 && nums[i + 1] !=
    5) {
                int swapPos =
    fiveLocations.get(fiveIndex);
                int temp = nums[i + 1];
                nums[i + 1] = 5;
                nums[swapPos] = temp;
                fiveIndex++;
            }
        }
        return nums;
    }
}
```